

المحاضرة الأولى:

مقدمة في الحواسيب والبرمجة

Introduction to Computers and Programming

في هذه المحاضرة سنتطرق إلى مقدمة في مجال علوم الحاسوب وأساسيات البرمجة، وحل المشكلات، وتصميم البرمجيات. يتم تغطية مكونات البرامج، مثل الكلمات المفتاحية، والمتغيرات، والعوامل، وعلامات الترقيم. كما يتم تقديم أدوات العمل، مثل Pseudocode، والمخططات الانسيابية (Flow Charts)، ومخططات التسلسل الهرمي (Hierarchy Charts).

Lecture Outlines:

- 1.1 Why Program?
- 1.2 Computer Systems: Hardware and Software
- 1.3 Programs and Programming Languages
- 1.4 What Is a Program Made of?
- 1.5 Input, Processing, and Output
- 1.6 The Programming Process
- 1.7 Procedural and Object-Oriented Programming

1.1 Why Program?

- **مفهوم:** يمكن لأجهزة الكمبيوتر القيام بالعديد من الوظائف المختلفة لأنها قابلة للبرمجة.
- **البرنامج:** هو مجموعة من التعليمات التي يتبعها الكمبيوتر لأداء مهمة معينة.
- **المبرمج، أو مطور البرمجيات:** هو الشخص الذي لديه التدريب والمهارات اللازمة لتصميم وإنشاء واختبار برامج الكمبيوتر.
- **برمجة الكمبيوتر:** هي فن وعلم في نفس الوقت.

- **يجب على المبرمجين:** تعلم لغات خاصة مثل لغة C++ لأن أجهزة الكمبيوتر لا تفهم اللغة الإنجليزية أو اللغات البشرية الأخرى. للغات مثل C++ قواعد صارمة يجب اتباعها بعناية.
- **طبيعة البرمجة الفنية والعلمية:** تجعل كتابة برامج الكمبيوتر تشبه تصميم السيارة: يجب أن تكون كل من السيارات والبرامج وظيفية، وفعالة، وقوية، وسهلة الاستخدام، وجميلة المظهر.

1.2 Computer Systems: Hardware and Software

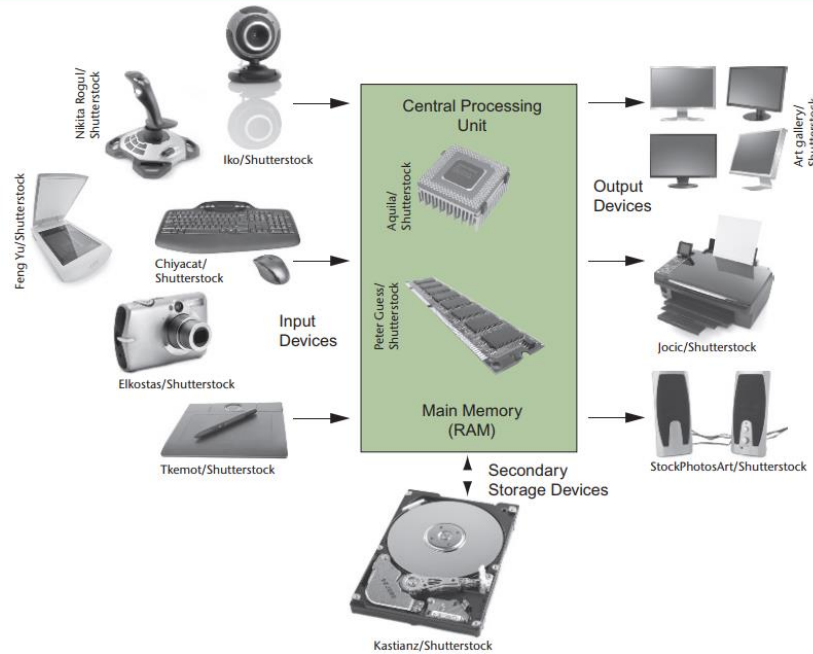
مفهوم: تتكون جميع أنظمة الكمبيوتر من أجهزة وبرمجيات متشابهة. توفر هذه القسم نظرة عامة على تنظيم أجهزة وبرمجيات الكمبيوتر القياسية.

الأجهزة (Hardware):

تشير الأجهزة إلى المكونات المادية التي يتكون منها الكمبيوتر. الكمبيوتر، كما نفكر فيه عادةً، ليس جهازًا فرديًا، ولكنه نظام من الأجهزة. مثل الآلات في الأوركسترا السمفونية، كل جهاز يؤدي دوره الخاص. يتكون نظام الكمبيوتر النموذجي من المكونات الرئيسية التالية:

- وحدة المعالجة المركزية (CPU)
- الذاكرة الرئيسية (Main Memory)
- أجهزة التخزين الثانوية (Secondary Storage Devices)
- أجهزة الإدخال (Input Devices)
- أجهزة الإخراج (Output Devices)

Figure 1-2 Typical devices in a computer system



وحدة المعالجة المركزية (CPU):

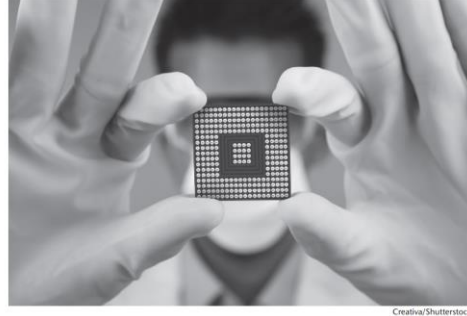
عندما يقوم الكمبيوتر بتنفيذ المهام التي يأمره البرنامج بها، نقول إن الكمبيوتر يعمل أو ينفذ البرنامج. وحدة المعالجة المركزية أو CPU هي الجزء من الكمبيوتر الذي يقوم فعلياً بتشغيل البرامج. تعد وحدة المعالجة المركزية المكون الأكثر أهمية في الكمبيوتر لأنه بدونها، لا يمكن للكمبيوتر تشغيل البرمجيات.

Figure 1-3 The ENIAC computer



- Electromechanical CPUs in early computers.
- Big CPU, was 8 feet tall, 100 feet long, and weighed 30 tons. (1945)

Figure 1-4 A microprocessor



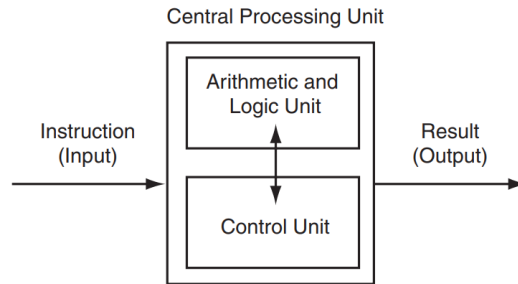
- CPUs are small chips known as microprocessors
- much more powerful

البرنامج هو سلسلة من التعليمات المخزنة في ذاكرة الكمبيوتر. عندما يعمل الكمبيوتر على تنفيذ برنامج، تكون وحدة المعالجة المركزية (CPU) منخرطة في عملية تُعرف رسمياً باسم دورة الجلب/فك التشفير/التنفيذ. خطوات دورة الجلب/فك التشفير/التنفيذ هي كما يلي:

1. **الجلب (Fetch):** تقوم وحدة التحكم في CPU بجلب التعليمات التالية في تسلسل تعليمات البرنامج من الذاكرة الرئيسية.
2. **فك التشفير (Decode):** تكون التعليمات مشفرة في شكل رقم. تقوم وحدة التحكم بفك تشفير التعليمات وتوليد إشارة إلكترونية.
3. **التنفيذ (Execute):** يتم توجيه الإشارة إلى المكون المناسب في الكمبيوتر مثل وحدة الحساب والمنطق (ALU)، أو محرك الأقراص، أو أي جهاز آخر. (تتسبب الإشارة في قيام المكون بتنفيذ عملية معينة).

تتكرر هذه الخطوات طالما هناك تعليمات لتنفيذها.

Figure 1-5 Organization of a CPU



الذاكرة الرئيسية (Main Memory):

تُعرف الذاكرة الرئيسية عادةً باسم ذاكرة الوصول العشوائي أو RAM سُميت بهذا الاسم لأن وحدة المعالجة المركزية (CPU) قادرة على الوصول بسرعة إلى البيانات المخزنة في أي موقع عشوائي في الذاكرة RAM، عادةً ما تكون الذاكرة RAM نوعاً من الذاكرة المتطايرة التي تُستخدم للتخزين المؤقت فقط أثناء تشغيل البرنامج. عند إيقاف تشغيل الكمبيوتر، يتم مسح محتويات الذاكرة RAM. داخل جهاز الكمبيوتر الخاص بك، يتم تخزين الذاكرة RAM في رقائق صغيرة.

يتم تقسيم ذاكرة الكمبيوتر إلى مواقع تخزين صغيرة تُعرف باسم البايتات. البايت الواحد يكفي لتخزين حرف واحد من الأبجدية أو رقم صغير. يتم تقسيم كل بايت إلى ثمانية مواقع تخزين أصغر تُعرف باسم البتات. مصطلح "بت" يشير إلى الرقم الثنائي (binary digit).

يتم تعيين رقم فريد لكل بايت يُعرف باسم العنوان. تكون العناوين مرتبة من الأصغر إلى الأكبر. يتم تحديد البايت بواسطة عنوانه بنفس الطريقة التي يتم بها تحديد صندوق البريد بواسطة عنوانه. يوضح الشكل 1-6 مجموعة من خلايا الذاكرة مع عناوينها. في الرسم التوضيحي، يتم تخزين بيانات عينة في الذاكرة. الرقم 149 مخزن في الخلية ذات العنوان 16، والرقم 72 مخزن في العنوان 23.

Figure 1-6 Memory

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29

أجهزة التخزين الثانوية (Secondary Storage Devices):

التخزين الثانوي هو نوع من الذاكرة يمكنه الاحتفاظ بالبيانات لفترات طويلة من الزمن، حتى في حالة عدم وجود طاقة للكمبيوتر. عادةً ما يتم تخزين البرامج في الذاكرة الثانوية ويتم تحميلها إلى الذاكرة الرئيسية عند الحاجة. يتم أيضاً حفظ البيانات المهمة مثل مستندات معالجة النصوص، وبيانات الرواتب.

أكثر أنواع أجهزة التخزين الثانوي شيوعاً هو محرك الأقراص (Hard Disc Drive: HDD). يقوم محرك الأقراص التقليدي بتخزين البيانات عن طريق تشفيرها مغناطيسياً على قرص دائري. أصبحت محركات الأقراص ذات الحالة الثابتة (Solid-State Drive: SSD)، التي تخزن البيانات في ذاكرة الحالة الثابتة، تحظى بشعبية متزايدة. لا تحتوي محركات الأقراص ذات الحالة الثابتة على أجزاء متحركة وتعمل بشكل أسرع من محركات الأقراص التقليدية. تحتوي معظم أجهزة الكمبيوتر على نوع من أجهزة التخزين الثانوي، سواء كان محرك أقراص تقليدياً أو محرك أقراص ذو حالة ثابتة، مثبتاً داخل علبتها. يمكن استخدام أجهزة التخزين الخارجية لإنشاء نسخ احتياطية من البيانات المهمة أو لنقل البيانات إلى كمبيوتر آخر. على سبيل المثال، محركات USB (الناقل التسلسلي العالمي) وبطاقات ذاكرة SD (الرقمية الأمانة) هي أجهزة صغيرة تظهر في النظام كمحركات أقراص. وهي غير مكلفة وموثوقة وصغيرة بما يكفي لحملها في جيبك.

أجهزة الإدخال (Input Devices):

الإدخال هو أي بيانات يجمعها الكمبيوتر من العالم الخارجي. يُطلق على الجهاز الذي يجمع المعلومات ويرسلها إلى الكمبيوتر اسم جهاز الإدخال. تشمل أجهزة الإدخال الشائعة لوحة المفاتيح، والفأرة، والشاشة التي تعمل باللمس، والمسح الضوئي، والكاميرا الرقمية، والميكروفون. يمكن أيضاً اعتبار محركات الأقراص، ومحركات الأقراص المضغوطة/دي في دي، ومحركات USB أجهزة إدخال لأنه يتم استرداد البرامج والمعلومات منها وتحميلها في ذاكرة الكمبيوتر.

أجهزة الإخراج (Output Devices):

الإخراج هو أي معلومات يرسلها الكمبيوتر إلى العالم الخارجي. قد يكون تقرير مبيعات، أو قائمة أسماء، أو صورة رسومية. يتم إرسال المعلومات إلى جهاز إخراج، والذي يقوم بتنسيقها وعرضها. تشمل أجهزة الإخراج الشائعة الشاشات، والطابعات، ومكبرات الصوت. يمكن أيضاً اعتبار أجهزة التخزين أجهزة إخراج لأن وحدة المعالجة المركزية ترسل إليها البيانات ليتم حفظها.

البرمجيات (Software):

إذا كان الكمبيوتر يعمل، فإن البرمجيات ليست اختيارية. كل ما يفعله الكمبيوتر، من لحظة تشغيله حتى إيقافه، يكون تحت سيطرة البرمجيات. هناك فئتان عامتان من البرمجيات:

- برمجيات النظام.
- برمجيات التطبيقات.

معظم برامج الكمبيوتر تنتمي بوضوح إلى إحدى هاتين الفئتين. دعونا نلقي نظرة أقرب على كل منهما.

برمجيات النظام (System Software):

البرامج التي تتحكم وتدير العمليات الأساسية للكمبيوتر تُعرف عموماً باسم برمجيات النظام. تشمل برمجيات النظام عادةً الأنواع التالية من البرامج:

- أنظمة التشغيل (Operating Systems):

نظام التشغيل هو مجموعة البرامج الأساسية على الكمبيوتر. يتحكم نظام التشغيل في العمليات الداخلية لأجهزة الكمبيوتر، ويدير جميع الأجهزة المتصلة بالكمبيوتر، ويسمح بحفظ البيانات واسترجاعها من أجهزة التخزين، ويسمح بتشغيل برامج أخرى على الكمبيوتر.

- برامج الأدوات المساعدة (Utility Programs):

تقوم برامج الأدوات المساعدة بمهام متخصصة تعزز عمل الكمبيوتر أو تحمي البيانات. من أمثلة برامج الأدوات المساعدة برامج مكافحة الفيروسات، وبرامج ضغط الملفات، وبرامج نسخ البيانات الاحتياطية.

- أدوات تطوير البرمجيات (Software Development Tools):

تُعرف الأدوات البرمجية التي يستخدمها المبرمجون لإنشاء البرمجيات وتعديلها واختبارها بأدوات تطوير البرمجيات. المترجمات (Compilers) وبيئات التطوير المتكاملة (Integrated Development Environments).

برمجيات التطبيقات (Application Software):

البرامج التي تجعل الكمبيوتر مفيداً للمهام اليومية تُعرف باسم برمجيات التطبيقات. هذه هي البرامج التي يقضي الأشخاص عادةً معظم وقتهم في تشغيلها على أجهزة الكمبيوتر الخاصة بهم مثل Microsoft Word، وهو برنامج معالجة نصوص، وMicrosoft PowerPoint، وهو برنامج عروض تقديمية. بعض الأمثلة الأخرى لبرمجيات التطبيقات تشمل برامج الجداول الحسابية، وبرامج البريد الإلكتروني، ومتصفحات الويب، وبرامج الألعاب.

1.3 Programs and Programming Languages

البرنامج هو مجموعة من التعليمات التي يتبعها الكمبيوتر لأداء مهمة معينة.

لغة البرمجة هي لغة خاصة تُستخدم لكتابة برامج الكمبيوتر.

ما هو البرنامج؟

تم تصميم أجهزة الكمبيوتر لاتباع التعليمات. برنامج الكمبيوتر هو مجموعة من التعليمات التي تخبر الكمبيوتر بكيفية حل مشكلة أو أداء مهمة. على سبيل المثال، لنفترض أننا نريد من الكمبيوتر حساب الراتب الإجمالي لشخص ما. إليك قائمة بالأشياء التي يجب أن يقوم بها الكمبيوتر:

1. عرض رسالة على الشاشة تسأل: "كم ساعة عملت؟"
2. الانتظار حتى يقوم المستخدم بإدخال عدد الساعات التي عملها. بمجرد أن يدخل المستخدم الرقم، يتم تخزينه في الذاكرة.
3. عرض رسالة على الشاشة تسأل: "كم تتقاضى في الساعة؟"
4. الانتظار حتى يقوم المستخدم بإدخال معدل الأجر في الساعة. بمجرد أن يدخل المستخدم الرقم، يتم تخزينه في الذاكرة.
5. ضرب عدد الساعات في معدل الأجر في الساعة، وتخزين النتيجة في الذاكرة.
6. عرض رسالة على الشاشة تخبر بمقدار المال الذي تم كسبه. يجب أن تتضمن الرسالة نتيجة الحساب الذي تم في الخطوة 5.

تُعرف هذه التعليمات مجتمعة باسم الخوارزمية. الخوارزمية هي مجموعة من الخطوات المحددة جيداً لأداء مهمة أو حل مشكلة.

على الرغم من أنك وأنا قد نفهم بسهولة التعليمات في خوارزمية حساب الأجر، إلا أنها ليست جاهزة للتنفيذ على الكمبيوتر. يمكن لوحدة المعالجة المركزية (CPU) في الكمبيوتر معالجة التعليمات المكتوبة بلغة الآلة فقط. إذا نظرت إلى برنامج بلغة الآلة، فسترى تدفقاً من الأرقام الثنائية (أرقام تتكون فقط من 1 و 0). تشكل الأرقام الثنائية تعليمات لغة الآلة، والتي تفسرها وحدة المعالجة المركزية على أنها أوامر. إليك مثال على كيف قد تبدو تعليمة بلغة الآلة:

1011010000000101

كما يمكنك أن تتخيل، فإن عملية ترميز خوارزمية بلغة الآلة هي عملية مملة وصعبة للغاية. بالإضافة إلى ذلك، فإن كل نوع من وحدات المعالجة المركزية له لغة الآلة الخاصة به. إذا كتبت برنامجاً بلغة الآلة للكمبيوتر "A" ثم أردت

تشغيله على الكمبيوتر "B"، الذي يحتوي على نوع مختلف من وحدة المعالجة المركزية، فسيتعين عليك إعادة كتابة البرنامج بلغة الآلة الخاصة بالكمبيوتر "B".

تم اختراع لغات البرمجة، التي تستخدم الكلمات بدلاً من الأرقام، لتسهيل مهمة البرمجة. يمكن كتابة برنامج بلغة برمجة مثل C++، والتي تكون أسهل في الفهم من لغة الآلة. يحفظ المبرمجون برامجهم في ملفات نصية، ثم يستخدمون برامج خاصة لتحويل برامجهم إلى لغة الآلة. يوضح البرنامج 1-1 كيف يمكن كتابة خوارزمية حساب الأجر بلغة C++.

Program 1-1

```
1 // This program calculates the user's pay.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     double hours, rate, pay;
8
9     // Get the number of hours worked.
10    cout << "How many hours did you work? ";
11    cin >> hours;
12
13    // Get the hourly pay rate.
14    cout << "How much do you get paid per hour? ";
15    cin >> rate;
16
17    // Calculate the pay.
18    pay = hours * rate;
19
20    // Display the pay.
21    cout << "You have earned $" << pay << endl;
22    return 0;
23 }
```

Program Output with Example Input Shown in Bold

```
How many hours did you work? 10  Enter
How much do you get paid per hour? 15  Enter
You have earned $150
```

لغات البرمجة:

بمعنى واسع، هناك فئتان من لغات البرمجة: لغات منخفضة المستوى ومرتفعة المستوى. اللغة منخفضة المستوى قريبة من مستوى وحدة المعالجة المركزية (CPU)، مما يعني أنها تشبه لغة الآلة الرقمية للكمبيوتر أكثر من اللغة

الطبيعية للبشر. أسهل اللغات التي يمكن للناس تعلمها هي اللغات عالية المستوى. تُسمى "عالية المستوى" لأنها أقرب إلى مستوى قابلية القراءة البشرية من قابلية القراءة الحاسوبية. يوضح الشكل 1-7 مفهوم مستويات اللغة.

Figure 1-7 Low-level versus high-level languages

High level (easily understood by humans)

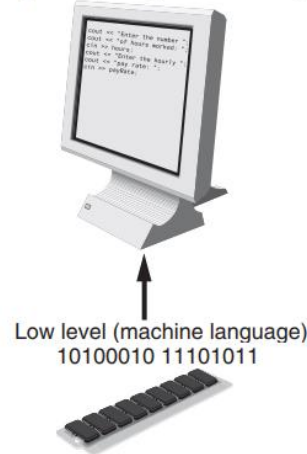


Table 1-1 Programming Languages

Language	Description
BASIC	Beginners All-purpose Symbolic Instruction Code. A general programming language originally designed to be simple enough for beginners to learn.
FORTRAN	Formula Translator. A language designed for programming complex mathematical algorithms.
COBOL	Common Business-Oriented Language. A language designed for business applications.
Pascal	A structured, general-purpose language designed primarily for teaching programming.
C	A structured, general-purpose language developed at Bell Laboratories. C offers both high-level and low-level features.
C++	Based on the C language, C++ offers object-oriented features not found in C. Also invented at Bell Laboratories.
C#	Pronounced "C sharp." A language invented by Microsoft for developing applications based on the Microsoft .NET platform.
Java	An object-oriented language that may be used to develop programs that run on many different types of devices.
JavaScript	JavaScript can be used to write small programs that run in webpages. Despite its name, JavaScript is not related to Java.
Python	Python is a general-purpose language created in the early 1990s. It has become popular in both business and academic applications.
Ruby	Ruby is a general-purpose language that was created in the 1990s. It is increasingly becoming a popular language for programs that run on web servers.
Visual Basic	A Microsoft programming language and software development environment that allows programmers to quickly create Windows-based applications.

الكود المصدري، الكود الهدف، والكود القابل للتنفيذ (Source Code, Object Code, and Executable Code):

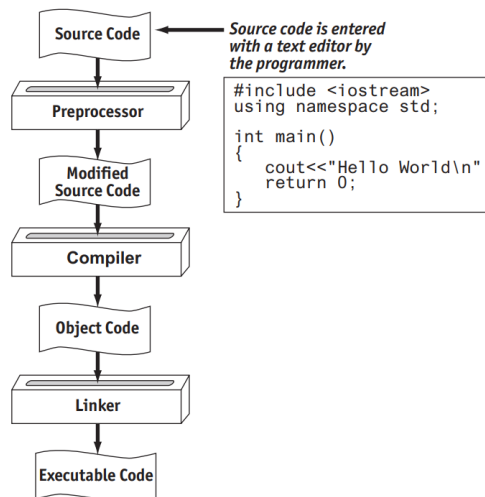
عند كتابة برنامج بلغة C++، يجب إدخاله إلى الكمبيوتر وحفظه في ملف. يتم استخدام محرر نصوص، يشبه برنامج معالجة النصوص، لهذه المهمة. تُعرف العبارات التي يكتبها المبرمج باسم الكود المصدري (Source Code)، ويُطلق على الملف الذي يتم حفظها فيه اسم الملف المصدري (Source File).

بعد حفظ الكود المصدري في ملف، تبدأ عملية ترجمته إلى لغة الآلة. خلال المرحلة الأولى من هذه العملية، يقرأ برنامج يسمى المعالج المسبق (Preprocessor) الكود المصدري. يبحث المعالج المسبق عن أسطر خاصة تبدأ بالرمز #. تحتوي هذه الأسطر على أوامر تجعل المعالج المسبق يقوم بتعديل الكود المصدري بطريقة ما.

خلال المرحلة التالية، يقوم المترجم (Compiler) بفحص الكود المصدري المعالج مسبقًا، ويقوم بترجمة كل تعليمة في الكود المصدري إلى تعليمة بلغة الآلة المناسبة. ستكشف هذه العملية أي أخطاء نحوية قد تكون موجودة في البرنامج. الأخطاء النحوية هي استخدامات غير صحيحة للكلمات المفتاحية، والعوامل، وعلامات الترقيم، وعناصر اللغة الأخرى. إذا كان البرنامج خاليًا من الأخطاء النحوية، يقوم المترجم بتخزين تعليمات لغة الآلة المترجمة، والتي تُسمى الكود الهدف (Object Code)، في ملف هدف (Object File).

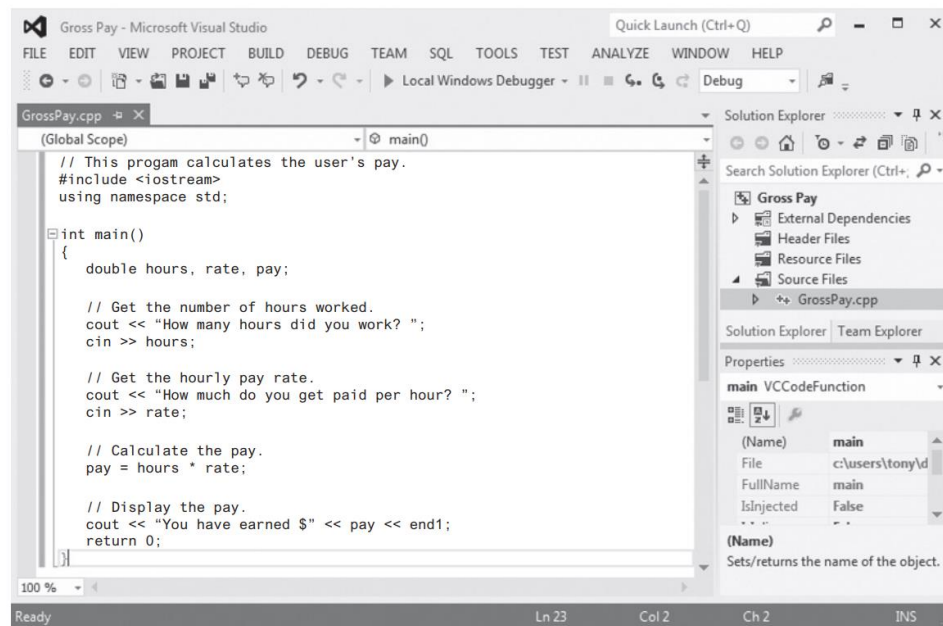
على الرغم من أن ملف الهدف يحتوي على تعليمات بلغة الآلة، إلا أنه ليس برنامجًا كاملاً. والسبب في ذلك هو أن لغة C++ مزودة بمكتبة تحتوي على كود مكتوب مسبقًا لأداء عمليات شائعة أو مهام قد تكون صعبة في بعض الأحيان. على سبيل المثال، تحتوي المكتبة على كود خاص بالأجهزة لعرض الرسائل على الشاشة وقراءة المدخلات من لوحة المفاتيح. كما توفر روتينات (توابع) للوظائف الرياضية، مثل حساب الجذر التربيعي لرقم. تُعرف هذه المجموعة من الكود باسم مكتبة وقت التشغيل (Runtime Library)، وهي واسعة النطاق. غالبًا ما تستخدم البرامج جزءًا منها. ومع ذلك، عندما يقوم المترجم بإنشاء ملف هدف، فإنه لا يتضمن كود الآلة لأي روتينات من مكتبة وقت التشغيل التي قد يكون المبرمج استخدمها. خلال المرحلة الأخيرة من عملية الترجمة، يقوم برنامج آخر يسمى الرابط (Linker) بدمج ملف الهدف مع روتينات المكتبة اللازمة. بمجرد انتهاء الرابط من هذه الخطوة، يتم إنشاء ملف قابل للتنفيذ (Executable File). يحتوي الملف القابل للتنفيذ على تعليمات بلغة الآلة، أو كود قابل للتنفيذ (Executable Code)، وهو جاهز للتشغيل على الكمبيوتر. يوضح الشكل 1-8 عملية ترجمة ملف مصدري بلغة C++ إلى ملف قابل للتنفيذ.

Figure 1-8 Translating a C++ source file to an executable file



تحتوي العديد من أنظمة التطوير، خاصة تلك الموجودة على أجهزة الكمبيوتر الشخصية، على بيئات تطوير متكاملة (IDEs). تتكون هذه البيئات من محرر نصوص، ومترجم، ومصحح أخطاء، وأدوات أخرى مدمجة في حزمة واحدة مع قوائم مشتركة. يتم تنفيذ المعالجة المسبقة، والترجمة، والربط، وحتى تشغيل البرنامج بنقرة زر واحدة، أو عن طريق اختيار عنصر واحد من القائمة. يوضح الشكل 1-9 شاشة من بيئة التطوير المتكاملة Microsoft Visual Studio.

Figure 1-9 An integrated development environment (IDE)



1.4 What Is a Program Made of?

المفهوم: هناك عناصر معينة مشتركة بين جميع لغات البرمجة.

عناصر اللغة: جميع لغات البرمجة تشترك في بعض الأشياء الأساسية. يوضح الجدول 1-2 العناصر المشتركة التي ستجدها في كل لغة برمجة تقريبًا.

لنلقي نظرة على بعض الأجزاء المحددة من البرنامج 1-1 (برنامج حساب الأجر) لرؤية أمثلة على كل عنصر مذكور في الجدول أعلاه. لتسهيل الأمر، تم إدراج البرنامج 1-1 مرة أخرى.

Table 1-2 Language Elements

Language Element	Description
Key Words	Words that have a special meaning. Key words may only be used for their intended purpose. Key words are also known as reserved words.
Programmer-Defined Identifiers	Words or names defined by the programmer. They are symbolic names that refer to variables or programming routines.
Operators	Operators perform operations on one or more operands. An operand is usually a piece of data, like a number.
Punctuation	Punctuation characters that mark the beginning or end of a statement, or separate items in a list.
Syntax	Rules that must be followed when constructing a program. Syntax dictates how key words and operators may be used, and where punctuation symbols must appear.

Program 1-1

```
1 // This program calculates the user's pay.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     double hours, rate, pay;
8
9     // Get the number of hours worked.
10    cout << "How many hours did you work? ";
11    cin >> hours;
12
13    // Get the hourly pay rate.
14    cout << "How much do you get paid per hour? ";
15    cin >> rate;
16
17    // Calculate the pay.
18    pay = hours * rate;
19
20    // Display the pay.
21    cout << "You have earned $" << pay << endl;
22    return 0;
23 }
```

الكلمات المفتاحية (الكلمات المحجوزة) (Key Words (Reserved Words):

تظهر ثلاث من الكلمات المفتاحية في لغة C++ في السطور 3 و5 وusing، namespace، int والكلمة double، التي تظهر في السطر 7، هي أيضاً كلمة مفتاحية في C++ هذه الكلمات، التي تكتب دائماً بأحرف صغيرة، لكل منها معنى خاص في لغة C++ ولا يمكن استخدامها إلا للأغراض المخصصة لها، يُسمح للمبرمج بابتكار أسماء خاصة به لأشياء معينة في البرنامج. ومع ذلك، فإن الكلمات المفتاحية محجوزة ولا يمكن استخدامها لأي شيء آخر غير الأغراض المحددة لها. جزء من تعلم لغة البرمجة هو تعلم الكلمات المفتاحية، معانيها، وكيفية استخدامها.



NOTE: The #include <iostream> statement in line 2 is a preprocessor directive.



NOTE: In C++, key words are written in all lowercase.

المُعرِّفات التي يحددها المبرمج (Programmer-Defined Identifiers):

الكلمات hours، rate، وpay التي تظهر في البرنامج في السطور 7، 11، 15، 18، و21 هي مُعرِّفات يحددها المبرمج. هذه الكلمات ليست جزءاً من لغة C++، بل هي أسماء ابتكرها المبرمج. في هذا البرنامج بالتحديد، هذه الأسماء تمثل متغيرات. المتغيرات هي أسماء مواقع في الذاكرة يمكنها تخزين البيانات.

العوامل (Operators):

في السطر 18، يظهر الكود التالي:

```
pay = hours * rate;
```

الرموز = و * كلاهما عوامل (Operators). فهي تقوم بإجراء عمليات على أجزاء من البيانات تُعرف باسم المعاملات (Operands). العامل * يقوم بضرب المعاملين الخاصين به، وهما في هذا المثال المتغيران hours و rate. أما الرمز = فيُسمى عامل الإسناد (Assignment Operator) فهو يأخذ قيمة التعبير الموجود على اليمين ويخزنها في المتغير الموجود اسمه على اليسار. في هذا المثال، يقوم العامل = بتخزين ناتج ضرب المتغير hours في المتغير rate داخل المتغير pay بعبارة أخرى، العبارة تقول: "اجعل المتغير pay يساوي hours مضروباً في rate"، أو " يتم إسناد قيمة hours مضروباً في rate للمتغير pay".

علامات الترقيم (Punctuation):

لاحظ أن السطور 3، 7، 10، 11، 14، 15، 18، 21، و 22 تنتهي جميعها بفاصلة منقوطة (;)، الفاصلة المنقوطة في لغة C++ تشبه النقطة في اللغة الإنجليزية فهي تُعلن نهاية الجملة الكاملة (أو العبارة، كما تُسمى في مصطلحات البرمجة). ومع ذلك، لا تظهر الفواصل المنقوطة في نهاية كل سطر في برنامج C++ هناك قواعد تحدد المواضع التي تكون فيها الفواصل المنقوطة مطلوبة والمواضع التي لا تكون فيها كذلك. جزء من تعلم لغة C++ هو تعلم مكان وضع الفواصل المنقوطة وعلامات الترقيم الأخرى.

الأسطر والعبارات (Lines and Statements):

غالبًا ما يُنظر إلى محتويات البرنامج من حيث الأسطر والعبارات. السطر هو ببساطة ما يظهر كسطر واحد في جسم البرنامج. يحتوي البرنامج 1-1 على 23 سطرًا. معظم هذه الأسطر تحتوي على شيء ذي معنى؛ ومع ذلك، بعض الأسطر فارغة. الأسطر الفارغة موجودة فقط لجعل البرنامج أكثر قابلية للقراءة.

العبارة (Statement) هي تعليمة كاملة تجعل الكمبيوتر ينفذ إجراءً معينًا. إليك العبارة التي تظهر في السطر 10 من البرنامج 1-1:

```
cout << "How many hours did you work? ";
```

هذه العبارة تجعل الكمبيوتر يعرض الرسالة "كم ساعة عملت؟" على الشاشة. يمكن أن تكون العبارات مزيّجا من الكلمات المفتاحية، والعوامل، والرموز التي يحددها المبرمج. غالبًا ما تشغل العبارات سطرًا واحدًا فقط في البرنامج، ولكن في بعض الأحيان تمتد إلى أكثر من سطر.

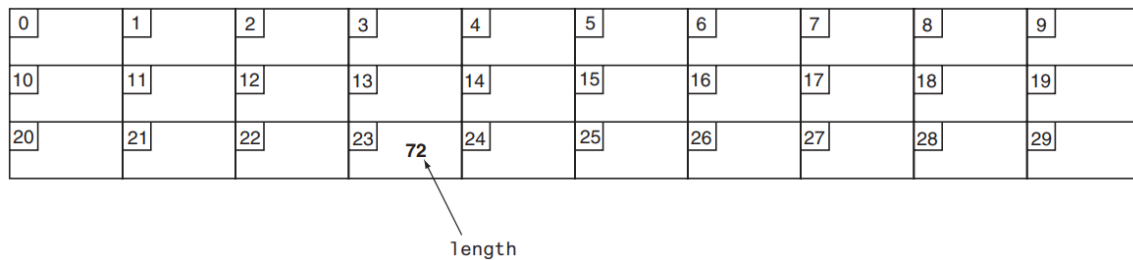
المتغيرات (Variables):

المتغير (Variable) هو موقع تخزين مُسمّى في ذاكرة الكمبيوتر يُستخدم لتخزين جزء من المعلومات. قد تتغير المعلومات المخزنة في المتغيرات أثناء تشغيل البرنامج (ومن هنا جاءت تسميتها "متغيرات"). لاحظ في البرنامج 1-1 أن الكلمات hours، rate، و pay تظهر في عدة أماكن. هذه الكلمات الثلاثة هي أسماء متغيرات. يُستخدم المتغير hours لتخزين عدد الساعات التي عملها المستخدم. بينما يخزن المتغير rate معدل أجر المستخدم في الساعة. ويحتفظ المتغير pay بناتج ضرب hours في rate، وهو إجمالي أجر المستخدم.

المتغيرات هي أسماء رمزية تمثل مواقع في ذاكرة الوصول العشوائي (RAM) في الكمبيوتر. عندما يتم تخزين المعلومات في متغير، يتم تخزينها فعليًا في ذاكرة الوصول العشوائي.

لنفترض أن هناك برنامجًا يحتوي على متغير اسمه `length` يوضح الشكل 10-1 الطريقة التي يمثل بها اسم المتغير موقعًا في الذاكرة. في الشكل 10-1، يحتفظ المتغير `length` بالقيمة 72. الرقم 72 مخزن فعليًا في ذاكرة الوصول العشوائي عند العنوان 23، ولكن الاسم `length` يمثل هذا الموقع بشكل رمزي. إذا كان ذلك مفيدًا، يمكنك التفكير في المتغير كصندوق يحتفظ بالمعلومات. في الشكل 10-1، الرقم 72 مخزن في الصندوق المسمى `length` حيث يمكن تخزين عنصر واحد فقط في الصندوق في أي وقت محدد. إذا قام البرنامج بتخزين قيمة أخرى في الصندوق، فإنها ستحل محل الرقم 72.

Figure 1-10 A variable name represents a memory location



تعريف المتغيرات (Variable Definitions):

في البرمجة، هناك نوعان عامان من البيانات: الأرقام والحروف. تُستخدم الأرقام لإجراء العمليات الحسابية، بينما تُستخدم الحروف لطباعة البيانات على الشاشة أو على الورق.

يمكن تصنيف البيانات الرقمية بشكل أكثر تفصيلاً. على سبيل المثال، الأعداد التالية هي أعداد صحيحة (Integers):

5

7

-129

32154

بينما الأعداد التالية هي أعداد حقيقية أو عشرية (Floating-Point Numbers):

3.14159

6.7

1.0002

عند إنشاء متغير في برنامج C++، يجب أن تعرف نوع البيانات التي سيتم تخزينها فيه. انظر إلى السطر 7 من البرنامج 1-1:

```
double hours, rate, pay;
```

الكلمة `double` في هذه العبارة تشير إلى أن المتغيرات `hours`، `rate`، و `pay` ستستخدم لتخزين أعداد عشرية ذات دقة مزدوجة (Double Precision Floating-Point Numbers)، تسمى هذه العبارة "تعريف المتغير" (Variable Definition)، يتم استخدامها لتحديد واحد أو أكثر من المتغيرات التي سيتم استخدامها في البرنامج ونوع البيانات التي ستخزنها. يتسبب تعريف المتغير في إنشاء المتغيرات في الذاكرة، لذا يجب تعريف جميع المتغيرات قبل استخدامها. إذا قمت بمراجعة قائمة البرنامج 1-1، ستلاحظ أن تعريفات المتغيرات تأتي قبل أي عبارات أخرى تستخدم هذه المتغيرات.

1.5 Input, Processing, and Output

المفهوم: الأنشطة الرئيسية الثلاثة للبرنامج هي الإدخال، المعالجة، والإخراج.

عادة ما تقوم برامج الكمبيوتر بعملية ثلاثية الخطوات تتمثل في جمع المدخلات، ومعالجة المعلومات التي تم جمعها، ثم إنتاج المخرجات.

- **الإدخال (Input):** هو المعلومات التي يجمعها البرنامج من العالم الخارجي. يمكن إرسالها إلى البرنامج من قبل المستخدم الذي يدخل البيانات عبر لوحة المفاتيح أو يستخدم الفأرة. يمكن أيضاً قراءتها من ملفات القرص أو الأجهزة المتصلة بالكمبيوتر. في البرنامج 1-1، يُسمح للمستخدم بإدخال جزأين من المعلومات: عدد ساعات العمل ومعدل الأجر في الساعة. يستخدم السطران 11 و15 الكائن cin يُنطق ("see in") لتنفيذ عمليات الإدخال هذه:

```
cin >> hours;
```

```
cin >> rate;
```

- **المعالجة (Processing):** بمجرد جمع المعلومات من العالم الخارجي، عادة ما يقوم البرنامج بمعالجتها بطريقة ما. في البرنامج 1-1، يتم ضرب عدد ساعات العمل في معدل الأجر في السطر 18، ويتم تعيين النتيجة للمتغير pay :

```
pay = hours * rate;
```

- **الإخراج (Output):** هو المعلومات التي يرسلها البرنامج إلى العالم الخارجي. يمكن أن تكون كلمات أو رسومات معروضة على الشاشة، أو تقرير يتم إرساله إلى الطابعة، أو بيانات مخزنة في ملف، أو معلومات يتم إرسالها إلى أي جهاز متصل بالكمبيوتر. في البرنامج 1-1، تقوم الأسطر 10 و14 و21 بتنفيذ عمليات الإخراج:

```
cout << "How many hours did you work? ";
```

```
cout << "How much do you get paid per hour? ";
```

```
cout << "You have earned $" << pay << endl;
```

تستخدم هذه الأسطر الكائن cout يُنطق ("see out") لعرض الرسائل على شاشة الكمبيوتر.

1.6 The Programming Process

المفهوم: عملية البرمجة تتكون من عدة خطوات، تشمل التصميم، الإنشاء، الاختبار، وتصحيح الأخطاء.

تصميم وإنشاء البرنامج (Designing and Creating a Program):

الآن بعد أن تعرفت على ما هو البرنامج، حان الوقت للنظر في عملية إنشاء البرنامج. في كثير من الأحيان، عندما يتم تكليف الطلاب غير الخبراء بمهام برمجية، يواجهون صعوبة في البدء لأنهم لا يعرفون ما يجب القيام به أولاً. إذا وجدت نفسك في هذه المعضلة، فقد تساعدك الخطوات المدرجة في الشكل 1-11. هذه هي الخطوات الموصى بها لكتابة البرنامج:

1. **فهم المشكلة:** تأكد من أنك تفهم تمامًا ما يتطلبه البرنامج.
2. **تخطيط الحل:** قم بتصميم خوارزمية أو خطة لحل المشكلة.
3. **كتابة الكود:** قم بتحويل الخطة إلى كود برمجي باستخدام لغة البرمجة المناسبة.
4. **اختبار البرنامج:** قم بتشغيل البرنامج واختباره للتأكد من أنه يعمل بشكل صحيح.

5. **تصحيح الأخطاء:** قم بتحديد الأخطاء وإصلاحها إن وجدت.

6. **تحسين الكود:** قم بتحسين الكود لجعله أكثر كفاءة وسهولة في القراءة.

باتباع هذه الخطوات، يمكنك تنظيم عملية تطوير البرنامج بشكل فعال وتجنب العديد من المشكلات الشائعة.

Figure 1-11 Steps for writing a program

1. Clearly define what the program is to do.
2. Visualize the program running on the computer.
3. Use design tools such as a hierarchy chart, flowcharts, or pseudocode to create a model of the program.
4. Check the model for logical errors.
5. Type the code, save it, and compile it.
6. Correct any errors found during compilation. Repeat Steps 5 and 6 as many times as necessary.
7. Run the program with test data for input.
8. Correct any errors found while running the program. Repeat Steps 5 through 8 as many times as necessary.
9. Validate the results of the program.

مثال على تصميم البرنامج:

باستخدام برنامج حساب الأجر كمثال، دعونا ننظر إلى كل خطوة من الخطوات بتفصيل أكبر.

1. تحديد ما يجب أن يفعله البرنامج بوضوح

تتطلب هذه الخطوة تحديد الغرض من البرنامج، والمعلومات التي سيتم إدخالها، والمعالجة التي سيتم، والمخرجات المطلوبة. دعونا نفحص كل من هذه المتطلبات لبرنامج المثال:

- **الغرض:** حساب إجمالي أجر المستخدم.
- **الإدخال:** عدد ساعات العمل، ومعدل الأجر في الساعة.
- **المعالجة:** ضرب عدد ساعات العمل في معدل الأجر في الساعة. النتيجة هي إجمالي أجر المستخدم.
- **الإخراج:** عرض رسالة توضح إجمالي أجر المستخدم.

من خلال تحديد هذه العناصر بوضوح، يمكنك إنشاء خريطة طريق تساعدك في كتابة البرنامج بشكل منظم وفعال.

2. تصور البرنامج وهو يعمل على الكمبيوتر

قبل إنشاء البرنامج على الكمبيوتر، يجب أولاً تصوره في ذهنك. تتمثل الخطوة الثانية في تصور البرنامج. حاول أن تتخيل كيف ستبدو شاشة الكمبيوتر أثناء تشغيل البرنامج. إذا كان ذلك مفيداً، ارسم صوراً للشاشة مع أمثلة للإدخال والإخراج في نقاط مختلفة من البرنامج. على سبيل المثال، هذه هي الشاشة التي ينتجها برنامج حساب الأجر:

كم عدد الساعات التي عملتها؟ 10

كم تتقاضى في الساعة؟ 15

لقد ربح 150 دولارًا

في هذه الخطوة، يجب أن تضع نفسك مكان المستخدم. ما هي الرسائل التي يجب أن يعرضها البرنامج؟ ما هي الأسئلة التي يجب أن يطرحها؟ من خلال معالجة هذه النقاط، ستكون قد حددت بالفعل معظم مخرجات البرنامج.

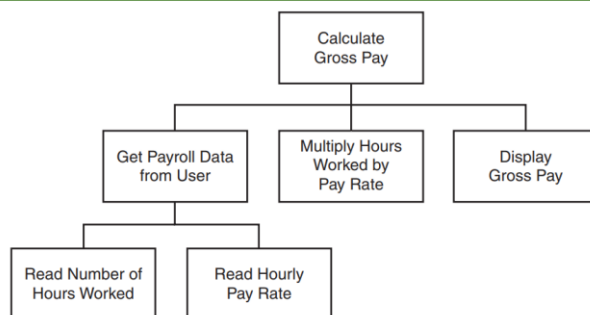
3. استخدام أدوات التصميم مثل المخطط الهرمي، المخططات الانسيابية، أو (Pseudocode) لإنشاء نموذج للبرنامج:

أثناء تخطيط البرنامج، يستخدم المبرمج واحدة أو أكثر من أدوات التصميم لإنشاء نموذج للبرنامج. ثلاث أدوات تصميم شائعة هي:

- **المخطط الهرمي (Hierarchy Chart):** هو رسم بياني يصور هيكل البرنامج بشكل مرئي. يحتوي على مربعات تمثل كل خطوة في البرنامج، حيث يتم توصيل المربعات بطريقة توضح العلاقة بينها. يوضح الشكل 12-1 مخططًا هرميًا لبرنامج حساب الأجر.
- **المخططات الانسيابية (Flowcharts):** تُستخدم لتمثيل تدفق البرنامج خطوة بخطوة باستخدام أشكال ورموز محددة.
- **الكود الزائف (Pseudocode):** هو وصف للبرنامج باستخدام لغة شبيهة باللغة البشرية، مما يساعد في تنظيم الأفكار قبل كتابة الكود الفعلي.

هذه الأدوات تساعد في تنظيم وتخطيط البرنامج قبل البدء في كتابة الكود، مما يجعل عملية التطوير أكثر كفاءة.

Figure 1-12 Hierarchy chart



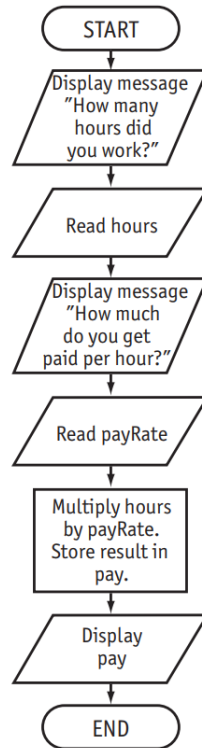
المخطط الانسيابي (Flowchart):

المخطط الانسيابي هو رسم بياني يوضح التدفق المنطقي للبرنامج. وهو أداة مفيدة لتخطيط كل عملية يقوم بها البرنامج والترتيب الذي يجب أن تحدث به هذه العمليات.

المخططات الانسيابية تستخدم أشكالاً ورموزاً محددة لتمثيل خطوات البرنامج، مثل:

- المستطيلات: لتمثيل العمليات.
- المعينات: لتمثيل قرارات أو شروط.
- الأسهم: لتمثيل اتجاه التدفق.

هذه الأداة تساعد في تصور كيفية تنفيذ البرنامج خطوة بخطوة، مما يسهل عملية التصميم والتحقق من صحة المنطق قبل كتابة الكود.



الكود الزائف (Pseudocode):

الكود الزائف هو مزيج بين اللغة البشرية ولغة البرمجة. على الرغم من أن الكمبيوتر لا يمكنه فهم الكود الزائف، إلا أن المبرمجين غالبًا ما يجدونه مفيدًا لكتابة الخوارزميات بلغة تشبه لغة البرمجة ولكنها قريبة من اللغة الطبيعية. على سبيل المثال، هذا هو الكود الزائف الذي يصف برنامج حساب الأجر:

- الحصول على بيانات الأجر.
- حساب إجمالي الأجر.
- عرض إجمالي الأجر.

على الرغم من أن الكود الزائف أعلاه يعطي نظرة عامة على البرنامج، إلا أنه لا يكشف عن جميع التفاصيل. إليك نسخة أكثر تفصيلاً من الكود الزائف:

1. ابدأ البرنامج.
2. اطلب من المستخدم إدخال عدد ساعات العمل.
3. اطلب من المستخدم إدخال معدل الأجر في الساعة.
4. اطلب من المستخدم إدخال معدل الأجر في الساعة.
5. اطلب من المستخدم إدخال معدل الأجر في الساعة.
6. اطلب من المستخدم إدخال معدل الأجر في الساعة.
2. اضرب عدد ساعات العمل في معدل الأجر لحساب إجمالي الأجر.
3. اعرض إجمالي الأجر للمستخدم.
4. انهي البرنامج.

الكود الزائف يساعد في تنظيم الأفكار وتبسيط المنطق قبل الانتقال إلى كتابة الكود الفعلي، مما يجعل عملية التطوير أكثر سلاسة.

4. التحقق من النموذج لوجود أخطاء منطقية (logical errors):

الأخطاء المنطقية هي أخطاء تتسبب في إنتاج البرنامج لنتائج خاطئة. بمجرد تجميع نموذج البرنامج باستخدام المخطط الهرمي، المخطط الانسيابي، أو الكود الزائف، يجب التحقق من وجود هذه الأخطاء. على المبرمج أن يتتبع الخطوات في المخططات أو الكود الزائف، ويتحقق من منطق كل خطوة. إذا تم اكتشاف خطأ، يمكن تصحيح النموذج قبل الانتقال إلى الخطوة التالية.

هذه الخطوة تساعد في ضمان أن البرنامج سيعمل بشكل صحيح ويحقق النتائج المرجوة دون أخطاء منطقية قد تؤثر على دقة المخرجات.

5. كتابة الكود، حفظه، وترجمته:

بمجرد إنشاء نموذج للبرنامج (مثل المخطط الهرمي، المخطط الانسيابي، أو الكود الزائف) والتحقق منه وتصحيحه، يكون المبرمج جاهزاً لكتابة الكود المصدري (Source Code) على الكمبيوتر. يقوم المبرمج بحفظ الكود المصدري في ملف ويبدأ عملية ترجمته إلى لغة الآلة. خلال هذه الخطوة، سيكشف المترجم (Compiler) عن أي أخطاء نحوية (Syntax Errors) قد تكون موجودة في البرنامج.

هذه الخطوة تعتبر تحويل التصميم النظري إلى برنامج فعلي قابل للتنفيذ.

6. تصحيح الأخطاء التي يتم اكتشافها أثناء الترجمة. كرر الخطوتين 5 و6 حسب الحاجة:

إذا أبلغ المترجم (Compiler) عن أي أخطاء، يجب تصحيحها. يجب تكرار الخطوتين 5 و6 حتى يصبح البرنامج خالياً من أخطاء الترجمة (Compile-Time Errors)، هذه الخطوة تضمن أن الكود المصدري خالٍ من الأخطاء النحوية وجاهز للتنفيذ.

7. تشغيل البرنامج باستخدام بيانات اختبارية كمدخلات:

بمجرد إنشاء ملف قابل للتنفيذ (Executable File)، يكون البرنامج جاهزاً لاختباره للكشف عن أخطاء وقت التشغيل (Runtime Errors). أخطاء وقت التشغيل هي أخطاء تحدث أثناء تنفيذ البرنامج، وعادة ما تكون أخطاء منطقية، مثل الأخطاء الحسابية.

لاختبار أخطاء وقت التشغيل، يجب تشغيل البرنامج باستخدام بيانات اختبارية أو مدخلات نموذجية. يجب أن تكون البيانات الاختبارية بحيث يمكن توقع المخرجات الصحيحة. إذا لم ينتج البرنامج المخرجات المتوقعة، فهذا يشير إلى وجود خطأ منطقي في البرنامج.

هذه الخطوة تساعد في ضمان أن البرنامج يعمل بشكل صحيح ويحقق النتائج المرجوة.

8. تصحيح أي أخطاء يتم اكتشافها أثناء تشغيل البرنامج. كرر الخطوات من 5 إلى 8 حسب الحاجة:

عند اكتشاف أخطاء وقت التشغيل (Runtime Errors) في البرنامج، يجب تصحيحها. يجب تحديد الخطوة التي حدث فيها الخطأ ومعرفة السبب. يمكن استخدام عملية الفحص اليدوي (Desk-Checking) للمساعدة في تحديد موقع الأخطاء.

الفحص اليدوي يعني أن المبرمج يبدأ بقراءة البرنامج أو جزء منه ويتتبع كل عبارة (Statement) خطوة بخطوة. غالباً ما يتم استخدام ورقة في هذه العملية لتسجيل المحتويات الحالية لجميع المتغيرات ورسم ما تبدو عليه الشاشة

بعد كل عملية إخراج. عندما تتغير محتويات متغير أو يتم عرض معلومات على الشاشة، يتم تسجيل ذلك. من خلال تتبع كل عبارة، يمكن اكتشاف العديد من الأخطاء وتصحيحها.

إذا كان الخطأ ناتجاً عن منطق غير صحيح (مثل صيغة رياضية خاطئة)، يجب تصحيح العبارة أو العبارات المتعلقة بالمنطق. إذا كان الخطأ ناتجاً عن فهم غير كامل لمتطلبات البرنامج، فيجب إعادة صياغة الغرض من البرنامج وتعديل المخطط الهرمي و/أو المخططات الانسيابية، الكود الزائف، والكود المصدري. بعد ذلك، يجب حفظ البرنامج، إعادة ترجمته، وإعادة اختباره. هذا يعني أنه يجب تكرار الخطوات من 5 إلى 8 حتى ينتج البرنامج نتائج مرضية بشكل موثوق.

9. التحقق من صحة نتائج البرنامج:

عندما تعتقد أنك قد قمت بتصحيح جميع أخطاء وقت التشغيل، أدخل بيانات اختبارية وتأكد مما إذا كان البرنامج يحل المشكلة الأصلية بشكل صحيح. هذه الخطوة الأخيرة تضمن أن البرنامج يعمل كما هو متوقع ويحقق الأهداف المحددة.

ما هي هندسة البرمجيات؟

مجال هندسة البرمجيات يشمل العملية الكاملة لتصميم وتطوير البرمجيات. يتضمن ذلك التصميم، الكتابة، الاختبار، تصحيح الأخطاء، التوثيق، التعديل، وصيانة مشاريع تطوير البرمجيات المعقدة. مثل المهندسين التقليديين، يستخدم مهندسو البرمجيات عدداً من الأدوات في عملهم. إليك بعض الأمثلة:

- مواصفات البرنامج (Program Specifications)
 - مخططات ورسوم توضيحية لمخرجات الشاشة (Charts and Diagrams of Screen Output)
 - المخططات الهرمية والمخططات الانسيابية (Hierarchy Charts and Flowcharts)
 - الكود الزائف (Pseudocode)
 - أمثلة على المدخلات المتوقعة والمخرجات المطلوبة (Examples of Expected Input and Desired Output)
 - برامج خاصة مصممة لاختبار البرامج (Special Software Designed for Testing Programs)
- معظم التطبيقات البرمجية التجارية كبيرة جداً. في كثير من الحالات، يتم تطويرها من قبل فريق أو أكثر من المبرمجين، وليس فرداً واحداً. من المهم أن يتم تحليل متطلبات البرنامج بدقة وتقسيمها إلى مهام فرعية يتم التعامل معها من قبل فرق فردية أو أفراد داخل الفريق.

في الخطوة الثالثة من عملية البرمجة، تم تقديم المخطط الهرمي كأداة للتصميم من الأعلى إلى الأسفل (Top-Down Design). المهام الفرعية التي يتم تحديدها في التصميم من الأعلى إلى الأسفل يمكن أن تصبح بسهولة وحدات (Modules) أو مكونات منفصلة للبرنامج. إذا كان البرنامج كبيراً جداً أو معقداً، يمكن تعيين فريق من مهندسي البرمجيات للعمل على الوحدات الفردية. مع تطور المشروع، يتم تنسيق الوحدات لتصبح في النهاية تطبيقاً برمجياً واحداً.

1.7 Procedural and Object-Oriented Programming

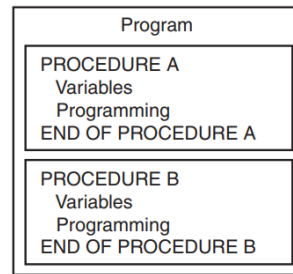
المفهوم: البرمجة الإجرائية (Procedural Programming) والبرمجة الكائنية (Object-Oriented Programming) هما طريقتان للتفكير في تطوير البرمجيات وتصميم البرامج.

لغة C++ هي لغة يمكن استخدامها للطريقتين لكتابة برامج الكمبيوتر: البرمجة الإجرائية والبرمجة الكائنية. في البرمجة الإجرائية، يقوم المبرمج ببناء إجراءات (Procedures) أو دوال (Functions)، كما تُسمى في C++. الإجراءات هي مجموعات من عبارات البرمجة التي تؤدي مهمة محددة. تحتوي كل إجراء على متغيراتها الخاصة، وغالبًا ما تشارك متغيرات مع إجراءات أخرى. يوضح الشكل 1-13 هذا المفهوم.

باختصار:

- البرمجة الإجرائية: تركز على تقسيم البرنامج إلى إجراءات أو دوال تقوم بمهام محددة.
- البرمجة الكائنية: تركز على تقسيم البرنامج إلى كائنات (Objects) تحتوي على بيانات ووظائف مرتبطة بها.

Figure 1-13 Procedures in a program

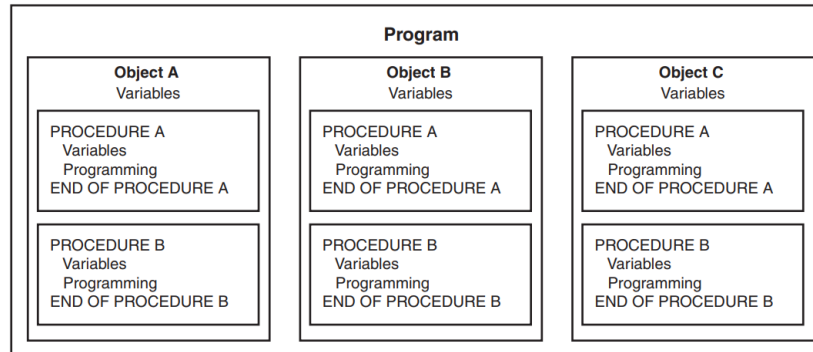


- البرمجة الإجرائية (Procedural Programming): تتمحور حول الإجراءات أو الدوال (Functions). يتم تنظيم البرنامج في شكل إجراءات تقوم بتنفيذ مهام محددة، وغالبًا ما تشارك هذه الإجراءات البيانات بينها.
- البرمجة الكائنية (Object-Oriented Programming - OOP): تتمحور حول الكائنات (Objects). الكائن هو عنصر برمجي يحتوي على البيانات (Data) والإجراءات التي تعمل على هذه البيانات. يعتبر الكائن وحدة مستقلة بذاتها. يوضح الشكل 1-14 هذا المفهوم.

باختصار:

- في البرمجة الإجرائية، التركيز على الإجراءات التي تعالج البيانات.
- في البرمجة الكائنية، التركيز على الكائنات التي تجمع بين البيانات والإجراءات في وحدة واحدة.

Figure 1-14 Objects in a program



تحتوي الكائنات، بداخلها، على كل من المعلومات والقدرة على معالجة هذه المعلومات. يتم تنفيذ العمليات على المعلومات داخل الكائن عن طريق إرسال رسالة إلى الكائن. عندما يستقبل الكائن رسالة تأمره بتنفيذ عملية معينة، فإنه ينفذ هذه التعليمات. باختصار:

- الكائنات تجمع بين البيانات والوظائف التي تعمل على هذه البيانات.
 - يتم التفاعل مع الكائنات من خلال إرسال رسائل إليها لتنفيذ مهام محددة.
- هذا النهج يجعل البرمجة الكائنية قوية ومرنة، خاصة عند التعامل مع أنظمة برمجية معقدة.

انتهت المحاضرة الأولى